behavior is that pointers to functions are *dangerous*. Such use is driven by a set of manual conventions. You have to remember to follow the convention to initialize those pointers. You have to remember to follow the convention to call all your functions through those pointers. If any programmer fails to remember these conventions, the resulting bug can be devilishly hard to track down and eliminate.

OO languages eliminate these conventions and, therefore, these dangers. Using an OO language makes polymorphism trivial. That fact provides an enormous power that old C programmers could only dream of. On this basis, we can conclude that OO imposes discipline on indirect transfer of control.

THE POWER OF POLYMORPHISM

What's so great about polymorphism? To better appreciate its charms, let's reconsider the example `copy` program. What happens to that program if a new IO device is created? Suppose we want to use the `copy` program to copy data from a handwriting recognition device to a speech synthesizer device: How do we need to change the `copy` program to get it to work with those new devices?

We don't need any changes at all! Indeed, we don't even need to recompile the `copy` program. Why? Because the source code of the `copy` program does not depend on the source code of the IO drivers. As long as those IO drivers implement the five standard functions defined by `FILE`, the `copy` program will be happy to use them.

In short, the IO devices have become plugins to the `copy` program.

Why did the UNIX operating system make IO devices plugins? Because we learned, in the late 1950s, that our programs should be *device independent*. Why? Because we wrote lots of programs that were device *dependent*, only to discover that we really wanted those programs to do the same job but use a different device.

For example, we often wrote programs that read input data from decks of cards,[6] and then punched new decks of cards as output. Later, our customers stopped giving us decks of cards and started giving us reels of magnetic tape. This was very inconvenient, because it meant rewriting large portions of the original program. It would be very convenient if the same program worked interchangeably with cards or tape.

The plugin architecture was invented to support this kind of IO device independence, and has been implemented in almost every operating system since its introduction.

Even so, most programmers did not extend the idea to their own programs, because using pointers to functions was dangerous.

OO allows the plugin architecture to be used anywhere, for anything.

DEPENDENCY INVERSION

Imagine what software was like before a safe and convenient mechanism for polymorphism was available. In the typical calling tree, main functions called high-level functions, which called mid-level functions, which called low-level functions. In that calling tree, however, source code dependencies inexorably followed the flow of control (Figure 5.1).

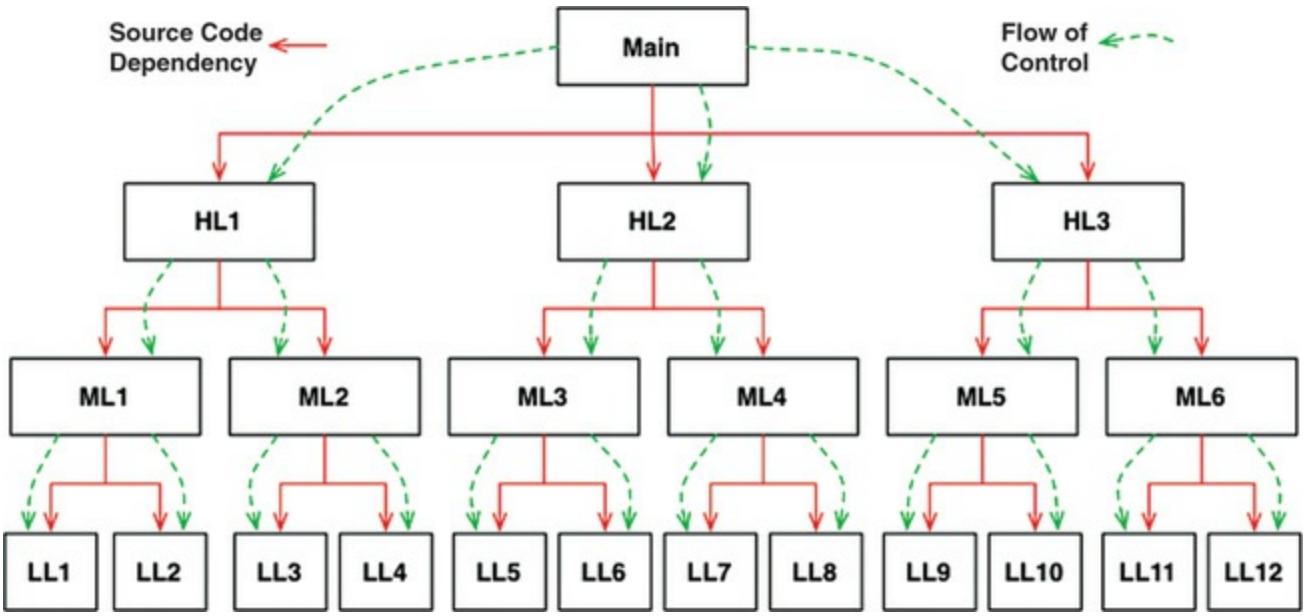

Figure 5.1 Source code dependencies versus flow of control

For `main` to call one of the high-level functions, it had to mention the name of the module that contained that function In C, this was a `#include`. In Java, it was an `import` statement. In C#, it was a `using` statement. Indeed, every caller was forced to mention the name of the module that contained the callee.

This requirement presented the software architect with few, if any, options. The flow of control was dictated by the behavior of the system, and the source code dependencies were dictated by that flow of control.

When polymorphism is brought into play, however, something very different can happen (Figure 5.2).

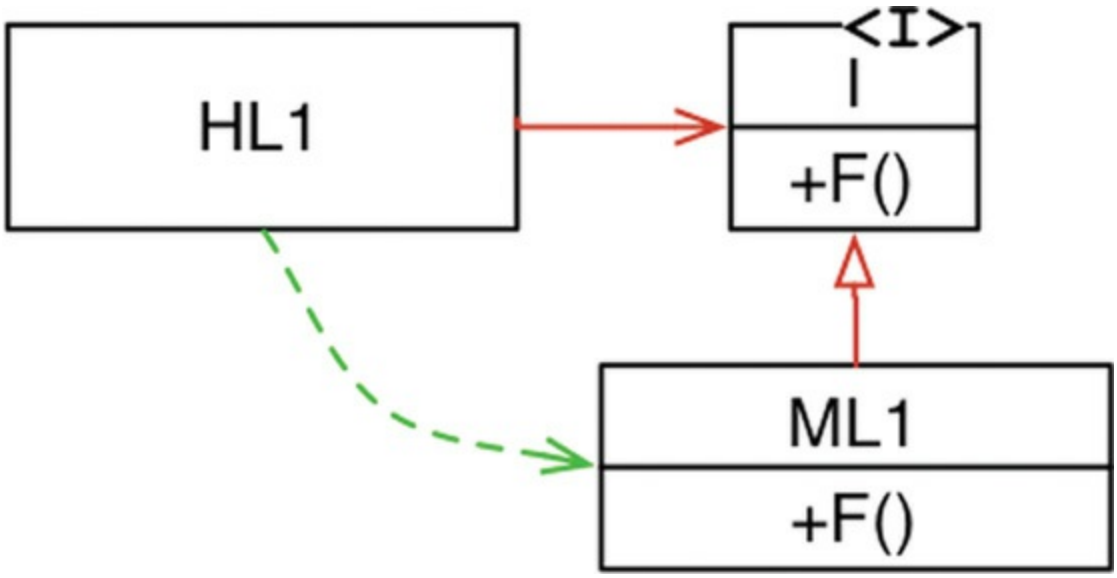

Figure 5.2 Dependency inversion

In Figure 5.2, module `HL1` calls the `F()` function in module `ML1`. The fact that it calls this function through an interface is a source code contrivance. At runtime, the interface doesn't exist. `HL1` simply calls `F()` within `ML1`.[7]

Note, however, that the source code dependency (the inheritance relationship) between `ML1` and the interface `I` points in the opposite direction compared to the flow of control. This is called *dependency inversion*, and its implications for the software architect are profound.

The fact that OO languages provide safe and convenient polymorphism means that *any source code dependency, no matter where it is, can be inverted*.

Now look back at that calling tree in Figure 5.1, and its many source code dependencies. Any of those source code dependencies can be turned around by inserting an interface between them.

With this approach, software architects working in systems written in OO languages have *absolute control* over the direction of all source code dependencies in the system. They are not constrained to align those dependencies with the flow of control. No matter which module does the calling and which module is called, the software architect can point the source code dependency in either direction.

That is power! That is the power that OO provides. That's what OO is really all about—at least from the architect's point of view.

What can you do with that power? As an example, you can rearrange the source code dependencies of your system so that the database and the user interface (UI) depend on the business rules (Figure 5.3), rather than the other way around.

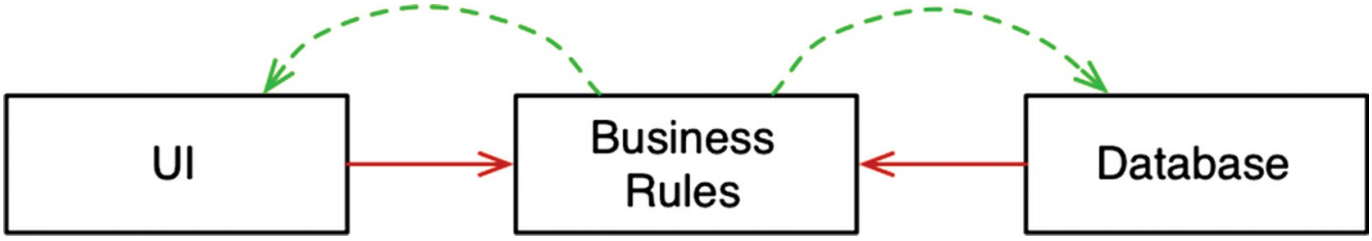

Figure 5.3 The database and the user interface depend on the business rules

This means that the UI and the database can be plugins to the business rules. It means that the source code of the business rules never mentions the UI or the database.

As a consequence, the business rules, the UI, and the database can be compiled into three separate components or deployment units (e.g., jar files, DLLs, or Gem files) that have the same dependencies as the source code. The component containing the business rules will not depend on the components containing the UI and database.

In turn, the business rules can be *deployed independently* of the UI and the database. Changes to the UI or the database need not have any effect on the business rules. Those components can be deployed separately and independently.

In short, when the source code in a component changes, only that component needs to be redeployed. This is *independent deployability*.

If the modules in your system can be deployed independently, then they can be developed independently by different teams. That's *independent developability*.

CONCLUSION

What is OO? There are many opinions and many answers to this question. To the software architect, however, the answer is clear: OO is the ability, through the use of polymorphism, to gain absolute control over every source code dependency in the system. It allows the architect to create a plugin architecture, in which modules that contain high-level policies are independent of modules that contain low-level details. The low-level details are relegated to plugin modules that can be deployed and developed independently from the modules that contain high-level policies.